STATEMENT OF OPERATION & TECHNICAL SPECIFICATION

For Registration of Computer Software under Copyright Act, 1957

Title of Work: Eventra 2026: Zero-Tamper Cryptographic QR Pass Issuance, Attendee Intake, and Gate Check-in Operating System

Author & Copyright Owner: Aditya Renake

Jurisdiction: Copyright Office, Government of India

---

1. ABSTRACT & PURPOSE

The computer program comprises a distributed full-stack software system architected to manage enterprise-scale technical hackathon operations. The system eliminates identity spoofing, credential fabrication, and race-condition gate entry through:

1. Deterministic HMAC-SHA256 digital signature computation embedded into dynamic 2D barcodes (QR passes).

2. Constant-time cryptographic verification at physical entry points.

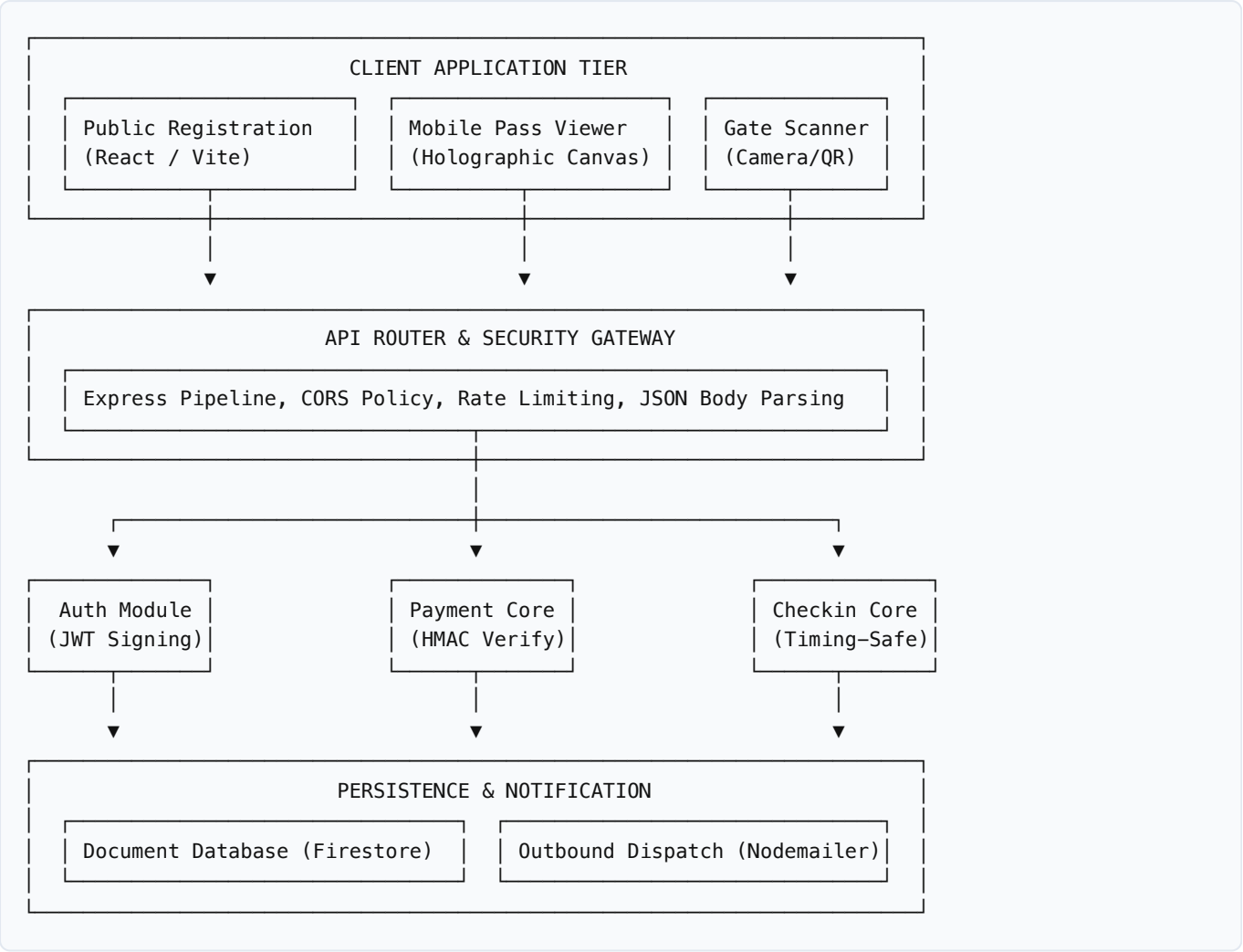
3. Automated payment escrow and order reconciliation via payment gateway hooks.

4. Real-time form intake transformation via cloud database synchronization.

5. In-browser camera visual gate check-in console with sub-100ms verification latencies.

---

2. ARCHITECTURAL SUBSYSTEMS & MODULE BREAKDOWN



Module 1: Cryptographic QR Signature Engine (`qrService.js`)

- **Mathematical Function:** Computes a truncated 16-hex-character Hash-Based Message Authentication Code (HMAC) using the SHA-256 cryptographic hash function over a concatenated canonical payload:

$$\text{Payload} = \text{UniqueId} + \text{"::"} + \text{AttendeeEmail} + \text{"::"} + \text{TicketTier}$$

$$\text{Signature} = \text{Truncate}_{16}(\text{HMAC-SHA256}(\text{SecretKey}, \text{Payload}))$$

- **Encoding:** Formats the generated signature into the standardized canonical QR token format `HS26.v1..`.

- **Security:** Renders passes immune to offline forgery, tampering, and unauthorized elevation of privileges.

Module 2: Timing-Safe Verification & Gate Check-in (`checkinController.js`)

- **Idempotent State Machine:** Validates scanned pass tokens using Node.js `crypto.timingSafeEqual()`, eliminating side-channel timing attacks.

- **Atomic Transition:** Transitions the registrant record from `CHECKED_IN: false` to `CHECKED_IN: true` with sub-millisecond timestamps, while raising instantaneous `ALREADY_CHECKED_IN` security warnings if a duplicate pass attempt is detected.

Module 3: Payment Gateway Order & Signature Reconciliation (`paymentController.js`)

- **Two-Phase Commit:**

- Phase 1: Initiates a localized order instance, synchronizing paise denomination with the Razorpay API.

- Phase 2: Intercepts client return credentials (`razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature`) and verifies authenticity against the server's cryptographic private key before writing to the database.

Module 4: Cloud Database Persistence Layer (`registrantRepo.js`, `staffRepo.js`)

- Provides unified CRUD operations, indexing, and transactional guarantees across distributed Google Cloud Firestore documents, with automatic fallback modes for isolated test runners.

Module 5: Client Presentation Layer (`RegistrationPage.jsx`, `DigitalPassPage.jsx`, `ScannerPage.jsx`)

- Built using React 18, utilizing the HTML5 Canvas API for holographic pass visual styling and `html5-qrcode` for multi-frame barcode scanning via webcam/mobile camera.

---

3. HARDWARE & OPERATING ENVIRONMENT

- **Target Processors:** x86_64, ARM64 (Apple Silicon, AWS Graviton).

- **Minimum Memory (RAM):** 512 MB (Optimal: 2 GB).

- **Supported OS:** Linux (Ubuntu 20.04+, Debian 11+), macOS 12+, Windows 10/11, Cloud Serverless (AWS Lambda, Vercel Functions).

- **Execution Engine:** Node.js v18.0.0 or higher.

- **Client Requirements:** Any modern ECMAScript 2020+ compatible web browser (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge) supporting HTML5 Camera MediaStream API.

---

4. ORIGINALITY & AUTHORSHIP STATEMENT

This computer program was conceived, structured, designed, and authored entirely as an original work by **Aditya Renake**. It contains original program algorithms, data schemas, and user interfaces and does not violate or infringe upon any pre-existing copyright.